P395: Guide 4

Start up a new notebook and title it 'Guide4_your_name'. Enter

```
import matplotlib.pyplot as plt

import numpy as np
```

Molecular Dynamics: Solving the dynamics of a system of particles

Imagine we have a system of interacting particles. How can we solve for their motion? We must solve Newton's 2nd law for every particle. For a particle *i*, the equation of motion is

$$m\frac{\mathrm{d}^2\vec{r_i}}{\mathrm{d}t^2} = \vec{F_i} = -\frac{\mathrm{d}}{\mathrm{d}\vec{r_i}}V(\vec{r_1}, \vec{r_2}, \ldots, \vec{r_N})$$

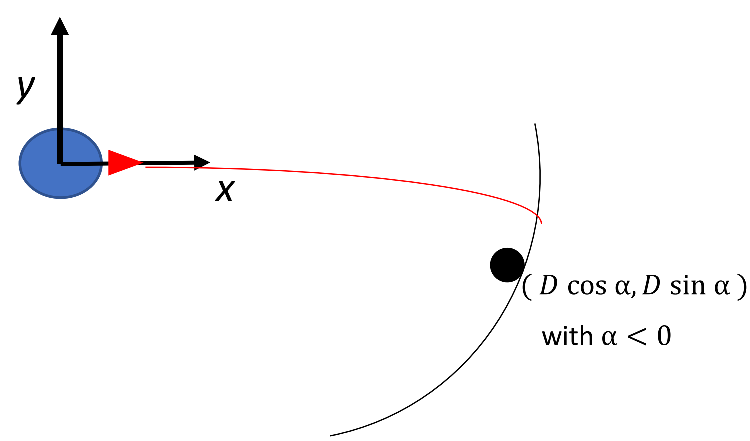
where $i = 1, \ldots, N$, and *N* is the number of particles. We assume that the force a particle experiences can be written in terms of the gradient of a potential $V(\{\vec{r}\})$ that depends on all the particle positions.

Python is too slow to simulate a large number of particles, so in this week's activity we are simply going to look at a 3-particle system (Earth, moon, and rocket) and see how to carry out the calculation of the trajectory of just the rocket but including the interactions of the moving earth and moon. So we will not be doing true molecular dynamics, or the methods that are used to implement it.

Task 1: Flight to the Moon: Apollo 8

Just over 50 years ago, we put a foot on the moon, and it looks like we'll be heading back there soon. Getting there was no small feat. In this week's activity guide, you will solve for the flight path of a rocket trying to get to the moon and back. Specifically, we want to do a fly-by of the moon: the rocket should go around the moon and then make its way back to earth. Such was the mission of Apollo 8, which was the first to go around the moon and return. Apollo 11 then went one step further and landed on the moon.

This problem is inspired from a similar problem from Dave Boal's Phys 395 course. There are 3 particles in our system: the Earth, the moon, and our spaceship (we'll ignore the effects of other objects like the Sun). We will only consider the motion of the rocket and moon around a fixed earth. A schematic is:



A rocket is launched out from the earth and must pass by the moon and hopefully return to earth. The three objects interact gravitationally via Newton's law of gravity. Recall, the force of gravity on object 2 (with mass m_2) due to object 1 (with mass m_1) is

$$\vec{F}_{12} = -\vec{r}_{12}\, G\, \frac{m_1\, m_2}{r_{12}^3}$$

where the displacement vector $\vec{r}_{12}$ points from 1 to 2, and the minus sign indicates that the force is in the opposite direction (i.e., attractive). Here $G = 6.6743015 \times 10^{-11}$ N m^2/kg^2. The mass of the earth is $m_E = 5.972 \times 10^{24}$ kg, and the mass of the moon is $m_M = 7.34767 \times 10^{22}$ kg.

1. Analysis: If we're considering just the motion of the rocket, why don't we need to know its mass? Put your answer in a text cell.

Simulation set up

The Earth is fixed at the origin. The moon moves counter-clockwise around the Earth on a circular path with radius $D = 3.84 \times 10^8$ m (it is actually pretty circular). When the rocket is launched at $t = 0$, the moon is at an angular position $\alpha < 0$ (in radians), so the initial (x,y) coordinates are $(D \cos \alpha, D \sin \alpha)$. It then carries out its uniform circular motion, going counter-clockwise around the Earth. From Kepler's laws, the period of the moon's orbit around the earth is $T^2 = 4\pi^2 D^3/Gm_\mathrm{E}$ and the corresponding angular frequency is $\omega^2 = Gm_\mathrm{E}/D^3$.

2. Use the above numbers to calculate T and ω for the moon.
3. Analysis: What equations do you need to use to calculate the moon's (x,y) location at $t > 0$? Put your answer in a text box.

You're going to start with your rocket on the surface of the earth and launch it radially outwards (as shown in the schematic above) with some initial velocity. The initial position of the rocket is $(R_E, 0)$ on the x-axis, for earth radius $R_E = 6.3781 \times 10^6$ m (the radius of the moon $R_M = 1.74 \times 10^6$ m). The initial velocity of the rocket is along the x-axis with values $(v_0, 0)$.

The goal of the simulation is to find values of α and v_0 so that the rocket just goes around the moon and loops back to earth. If the velocity is too large, the rocket overshoots and heads off into space; if the velocity is too small, the rocket never makes it to the moon.

An estimate of the approximate speed that you need at launch is to consider the situation where it makes it out to the moon's orbit. Conservation of energy dictates that

$$\frac{1}{2} m v_0^2 - \frac{G m_\mathrm{E} m}{R_\mathrm{E}} = -\frac{G m_\mathrm{E} m}{D}$$

where we've ignored the potential energy due to the interaction with the moon.

4. Using the values for the various masses and distances, calculate an estimate for the launch velocity to just reach the moon's orbit using the equation above. (This will be a good starting guess.)

Recall from last week's activity, that to solve a 2nd-order ordinary differential equation (ODE), one writes it down as a system of 1st-order ODEs. The motion of the rocket in 2D corresponds to solving a set of 4 first-order ODEs: two from $\mathrm{d}\vec{v}/\mathrm{d}t = \vec{f}$ and two from $\mathrm{d}\vec{r}/\mathrm{d}t = \vec{v}$.

5. Code a function that returns the 4 derivatives needed for solving the rocket's ODE which can be passed to `solve_ivp`. It should return both the acceleration (*i.e.*, dv/dt for both x,y components) acting on the rocket at time t due to the gravitational forces and the rocket's velocity (*i.e.*, dr/dt for both x,y components).
6. Check that your function is working properly. If you input the rocket's initial position of (R_E, 0) and velocity (v_0, 0) and have the location of the moon at (D, 0), what acceleration does your function return?
7. Does it match the magnitude and direction you know it should be? (HINT: what do you know to be the acceleration due to gravity at the surface of the earth?)

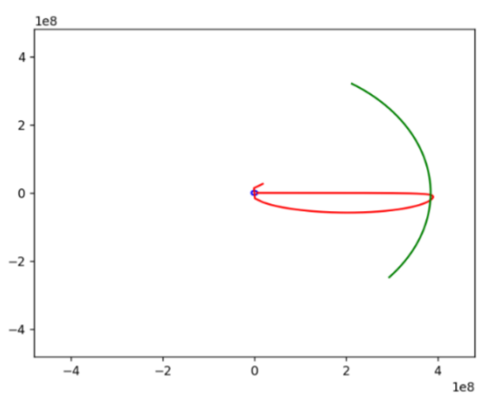
Task 2: Calculate Rocket Trajectories

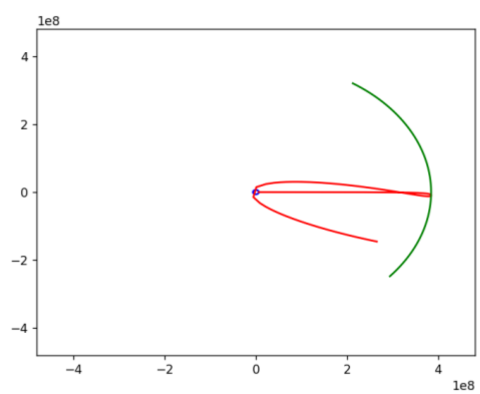
Now you'll use `solve_ivp` to find the trajectory of the rocket at different values of α and v_0. You can let it use adaptive step sizes at first to rapidly calculate single trajectories.

From your simulations, this is what you should calculate:

Explore: First, we'll explore the system dynamics so you can get a sense of how sensitive the rocket's trajectory is to initial conditions.

1. With α = -0.8 radians, find a few v_0 that allow you to go past the moon and return to Earth (say within $2R_E$). To start, use the value of v_0 from your result in (Task 1.4) above and use `solve_ivp` to find the rocket trajectory. Plot the trajectories of the moon and rocket. You may also need to play around with your final time t_f. Then change v_0 in small increments/decrements of 1-5 m/s and look at the trajectories until you see some successful return trips. Here are some examples of good trajectories:





2. Analysis: Now if you change α, what do you think will happen to your range of v_0? For example if you make α more negative, will your v_0 shift to be slower or faster? Give your answer and reasoning in a text box.

Task 3: Animating your trajectory

Please try the following task as it can be a very helpful tool. I recommend switching back to the Classic Jupyter environment on Syzygy as the following code has been tested there. If you cannot get the animation to work, do not spend significant time on it and move on to Task 4; you will not be penalized if you cannot get Task 3 to work.

Something that can be helpful when carrying out dynamical simulations is to animate the motions of your particles. This allows one to see how the dynamics is evolving and whether there might be any potential bugs in the code, as indicated by weird behaviour in the motion. Animating things in python is (sort of) straightforward. It involves creating a plot (that's your zeroth frame), and then having an update function that updates the plot as you move forward through the frames. There are probably several different ways to animate the rocket's trajectory, but the code I provide below initiates a line plot of the trajectory, and then updates the line by adding one new coordinate at each frame. It also makes a scatter plot for the location of the moon.

1. First, we need a trajectory to animate. Choose an α and v_0 and use `solve_ivp` to find the solution at a uniformly spaced set of time points using `t_eval` (use a reasonable dt to get a few thousand points given the trip takes on the order of a few days to complete). Use the solution to make a 2D array of the (x,y) points for the rocket's trajectory. Also create a 2D array of points for the trajectory of the moon using the same set of time points. Both arrays should be of shape (N_T, 2) where N_T is the number of time points.
2. Turn on interactive mode with `%matplotlib widget`.
3. Using your arrays of positions for the rocket and moon, try the animation code available at Canvas->Notes->Guide4_Animation.ipynb (you may need to play with the interval to get the animation running at a decent speed).
4. Use your animation code to look at several trajectories with different α and v_0. Does it help you to get a better intuition for the dynamics? Animate an unsuccessful trajectory and a successful one.
5. Analysis: From your animations, describe in words how the motion of the rocket changes over the course of each trajectory. Put your answer in a text box.
6. Turn the interactive mode off: `%matplotlib inline`

Task 4: Map out launch strategies

You should now have some intuition for how α and v_0 affect the rocket's trajectory. In the following task you will systematically map out the 'quality' of the trajectory as measured by different metrics (*e.g.*, distance of closest approach to the moon) at different α and v_0.

1. For each fixed value of α between -0.6 and -1.0 (say with step size 0.1), find the range of v_0 that allows you to go to the moon and back. For each successful v_0, record the closest distance the rocket got to the moon [NOTE: the rocket has to go past the moon ($r > D$) and loop back] and the total time the rocket took to return to earth (again, $r <= 2R_E$).
2. Make plots of the metrics you calculated in (1). For example, you might plot the smallest successful v_0 at each α. You can also plot the distance of closest approach vs. v_0 at a given α. Lastly, you could plot the minimum return trip time versus α.

3. Analysis: From your calculations and plots, what would you estimate to be the largest $|\alpha|$ that the moon could start at that would still allow your rocket to make a successful trip?
4. Discussion: From your analysis, try to come up with some recommendations for how you would launch your rocket to minimize the time (less food to pack) and weight (lower speed means less fuel, or conversely more fuel to use in case of emergency). Is there a best α and v_0?

Task 5: Wrap up

1. Please provide any feedback on questions that you found difficult to understand because of the wording.
2. Please provide any other comments about this notebook.